Ingeniería del Software 3

Clase 9 · Proteger y vigilar lo que ya está en producción

UCC · Ingeniería en Sistemas · 2026 · Ing. Ariel Schwindt

Lo que vas a ver en este video

tu repositorio
es público: lo de seguridad es gratis

tu producción, viva
la de siempre, desde que la creaste

dos cuentas nuevas
un monitor, y Sentry · gratis, sin tarjeta

El orden del video

1 · Proteger	2 · Vigilar	3 · Cuando algo falla	4 · El práctico
tres protecciones automáticas de GitHub	si está viva, y cómo se comporta	qué se pregunta un equipo, y cómo se mide	qué hacés · qué entregás

Se hace sobre lo que ya tenés: no se crea ningún servicio nuevo en Render ni en Neon

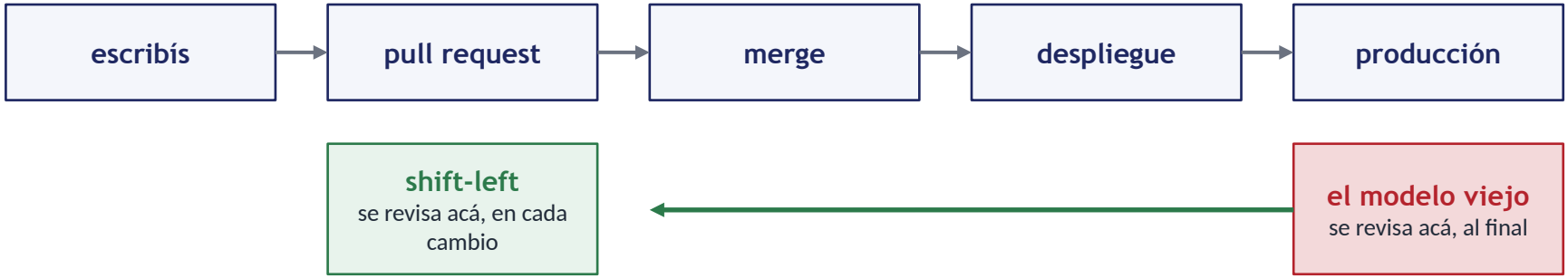
Dos situaciones que le pasan a todos los equipos

una clave casi llega al repositorio	producción se cae un sábado
alguien está por subir un archivo con una clave adentro · un compañero la ve en la revisión y lo frena	nadie se entera hasta que un usuario avisa, cuarenta minutos después
¿y si no la veía?	**¿y si nadie avisaba?**

En los dos casos el equipo dependió de la suerte: de que alguien mirara, de que alguien avisara

Hoy: una protección que frena antes de que entre, y un vigilante que avisa cuando se cae

Shift-left: revisar la seguridad antes, no al final



por qué
cuanto más tarde aparece un problema, más cuesta arreglarlo — el argumento del práctico 4, aplicado a la seguridad

cómo
automático, en cada pull request, igual que corrés los tests

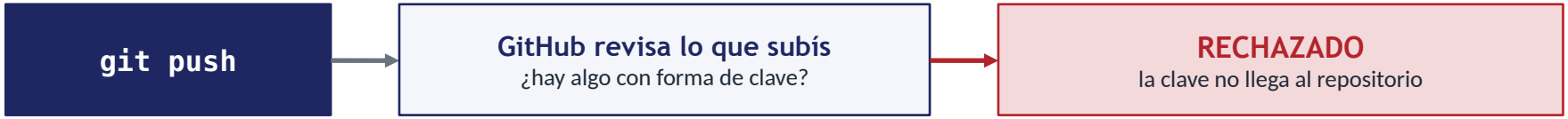
DevSecOps: la seguridad como una verificación más de tu pipeline

Tres protecciones, y cada una mira una cosa distinta

1 · Secretos	2 · Dependencias	3 · Tu código
que una clave no llegue al repositorio	el código que usás y no escribiste	errores de seguridad en lo que sí escribiste
la herramienta	la herramienta	la herramienta
`push protection`	`Dependabot`	`CodeQL`

Las tres son de GitHub, y en un repositorio público son gratis

1 · Secretos: el push que no entra




```
$ git push origin mi-rama
remote: error: GH013: Repository rule violations found
remote: - Push cannot contain secrets
```

qué reconoce
claves con un formato conocido: de GitHub, de Amazon, de cientos de servicios

qué no reconoce
una contraseña inventada por vos: no tiene forma de nada, y pasa

No avisa después: frena antes — y las claves siguen sin escribirse en el código

2 · Dependencias: el código que no escribiste

el código que ejecuta tu aplicación

lo que escribiste	tus dependencias — y las dependencias de tus dependencias

se publica una falla
en una librería que usás

→ **Dependabot te avisa**
compara contra un catálogo público

→ **te abre un pull request**
que cambia tu versión por una corregida

→ **tu pipeline lo verifica**
como a cualquier otro

→ **mergeás**
con confianza

Además, con un archivo corto, te propone cada semana versiones nuevas, haya falla o no

Tus tests, tu cobertura y tus pruebas de punta a punta son lo que te deja mergear sin miedo

3 · Tu código: leerlo sin ejecutarlo

tus tests
ejecutan el programa y miran si el resultado es el esperado

CodeQL
lee el código sin ejecutarlo, y busca formas de escribir que se sabe que son peligrosas

el ejemplo clásico: un dato del usuario, pegado adentro de una consulta

```
var sql = "SELECT * FROM tareas WHERE titulo = '" + titulo + "'";
```

se llama inyección
quien escribe ese dato puede hacerle decir a tu consulta lo que quiera

OWASP Top 10
la lista de las diez clases de fallas más comunes en aplicaciones web: sirve para ponerle nombre a cada hallazgo

Lo que encuentra aparece en la pestaña Security de tu repositorio

Prenderlas cuesta poco

protección	qué hay que hacer	dónde
`push protection`	ya está activa en tu repositorio	sólo verificás que lo esté: Settings → Advanced Security
`Dependabot`	un archivo y dos opciones	.github/dependabot.yml, en main · alertas y actualizaciones de seguridad, en Settings
`CodeQL`	dos clicks	Settings → Advanced Security → CodeQL → configuración por defecto

El paso a paso de cada una está en la guía

¿Cómo sabés que tu producción está viva?

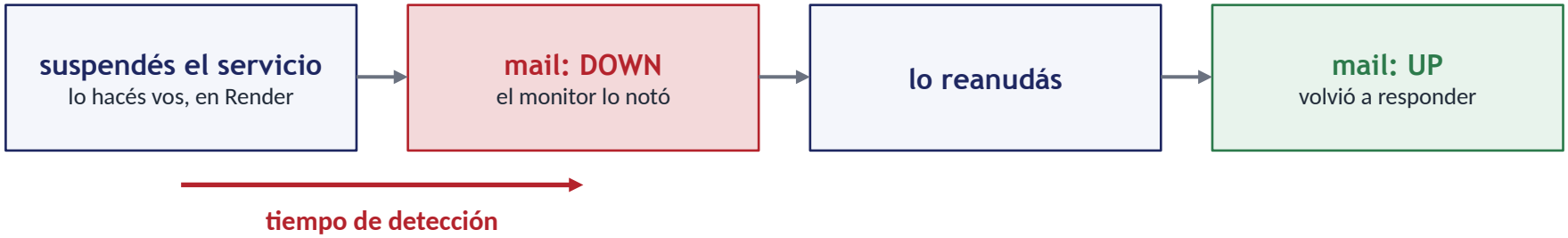
«Nadie se quejó» no es saberlo



por qué externo
si corriera en tu mismo servidor, se caería junto con lo que tiene que vigilar

Un monitor: alguien de afuera que pregunta, y que avisa cuando no le contestan

La caída, medida: DOWN, UP y el tiempo



con el monitor
minutos

sin el monitor
lo que tarde un usuario en avisarte

En el práctico la caída la provocás vos, y los dos avisos quedan en el historial del monitor

Una alerta sirve cuando pide que alguien haga algo

alerta que sirve	alerta que sobra
producción está caída → alguien tiene que hacer algo ya	un despliegue terminó bien → nadie tiene que hacer nada

mails por todo → dejás de leerlos → llega el importante → se pierde entre los demás

a eso se le dice fatiga de alertas

La regla: pocas alertas, y que cada una pida una acción

Un cuidado: el monitor gasta tu cuota de Render

tu servicio gratuito
se duerme a los 15 minutos sin visitas

→

el monitor
lo visita cada 5 minutos

→

resultado
no se duerme nunca

un servicio despierto todo el mes
720 a 744 horas

tu cuota, para TODOS tus servicios
750 horas por mes

Si te pasás, Render suspende todos tus servicios gratuitos — producción incluida

La regla: prendés el monitor para el ejercicio, y lo PAUSÁS al terminar — el historial queda

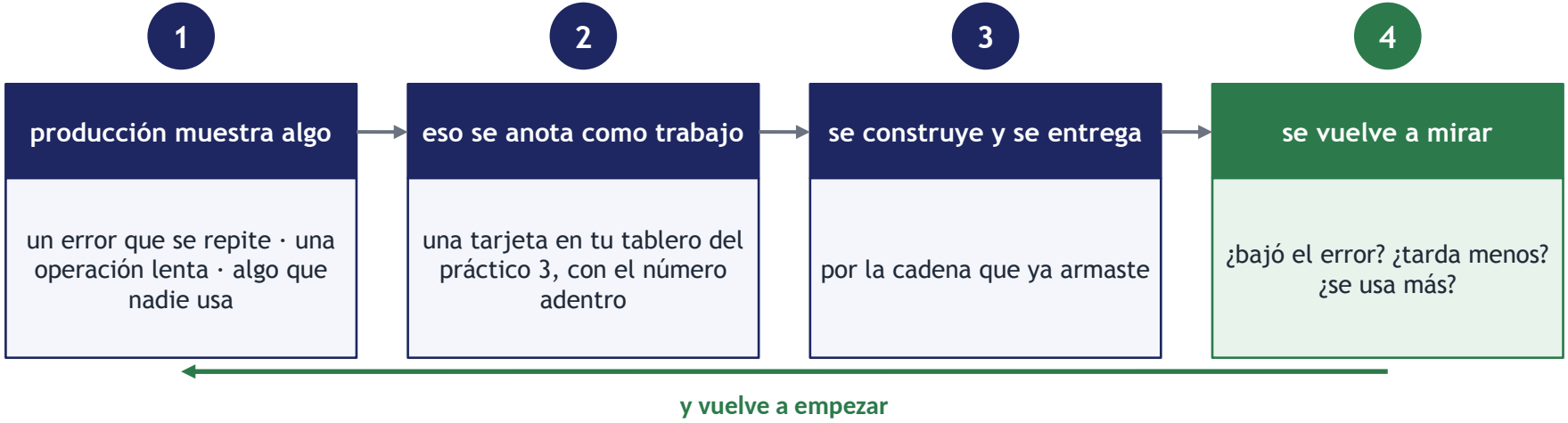
Estar viva no es andar bien

El monitor sólo dice si tu aplicación contesta. Para saber cómo se comporta, tiene que contarlo ella

la pregunta	qué te muestra	medido en la app de la cátedra
¿qué falla?	cada error, con el archivo y la línea, y cuántas veces pasó	`1 error · 2 veces · en GET /api/falla`
¿qué tarda?	cuánto demora cada operación, y cada consulta a la base	`listar tareas: hasta 1,4 segundos`
¿qué se usa?	cuántas veces se pide cada operación	`listar: 14 · agregar: 4 · borrar: 1`

Se hace con una librería en tu código, que manda esos datos a un servicio: acá, Sentry

Del uso a la próxima versión



Sin ese circuito, la próxima versión se decide por intuición: con él, por lo que pasó

Cuando algo se rompe: ¿qué pregunta se hace?

«¿quién fue?»
la próxima vez, la gente esconde lo que pasó
el equipo no sabe cómo falla su sistema

«¿qué permitió que pasara?»
aparece lo que le faltaba al sistema
y se arregla para todos

el mismo incidente, contado de las dos formas

«Juan subió una clave»
no arregla nada

«no había nada que frenara una clave en un push»
la solución es una protección

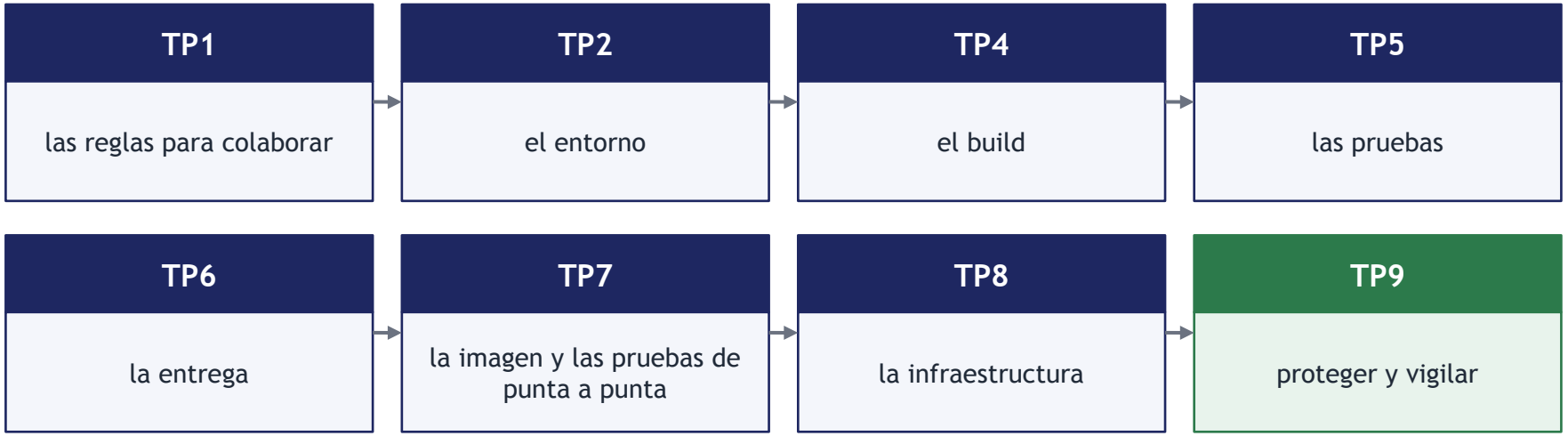
Ese análisis escrito, sin culpables, se llama postmortem blameless

Medirse: las cuatro métricas de la Clase 1

cada cuánto	cuánto tarda	cuántos salen mal	cuánto tardás
llevás cambios a producción	un cambio, desde que lo escribís hasta que está en producción	de todos los despliegues que hiciste	en recuperarte cuando uno sale mal
de dónde sale	de dónde sale	de dónde sale	de dónde sale
tus releases	**las fechas del commit y del despliegue**	**tu historial de despliegues**	**tu vuelta atrás del práctico 6**

En la Clase 1 eran números de otras empresas: ahora podés calcular los tuyos

El recorrido de la Clase 1, completo



Cada pieza es un archivo en tu repositorio o una configuración que podés mostrar

TP9 — seis tareas cortas

1	**Dependencias vigiladas**	Dependabot activo · un pull request de actualización mergeado, con tu pipeline en verde
2	**Tu código, escaneado**	CodeQL activo, con su primer análisis terminado
3	**Los secretos, con guardia**	verificar que push protection esté prendida
4	**Producción observada**	el monitor sobre tu /health · una caída provocada por vos en su historial · y pausado al terminar
5	**Tu aplicación, observada**	Sentry en tu back y tu front · un error y tu uso real a la vista · una tarjeta en tu tablero nacida de un dato
6	**Release v9.0.0**	el cierre del práctico, como siempre

Práctica recomendada — no se entrega

ver el push bloqueado · analizar dos hallazgos de CodeQL · calcular tus cuatro métricas · escribir el postmortem. De ahí salen las preguntas de la defensa

TP9 — Entregables

formulario	decisiones.md	no se pega nada
sólo la URL de tu repositorio	tres cosas: qué actualizaba el pull request que mergeaste, y por qué · cuánto tardó tu monitor, y tu operación más usada y la más lenta · problemas y uso de IA	ni capturas, ni salidas de comandos, ni evidencias.md

En la defensa, en vivo: el recorrido

Security con las tres protecciones · tu pull request de Dependabot · tu monitor con el DOWN y el UP · tu Sentry con un error, lo más lento y lo más usado · tu tarjeta

es la defensa de los prácticos 5 a 9

25% técnico	75% defensa

Por dónde empezar: dos cosas tardan solas

1	**Dependabot**	primero: el primer pull request puede tardar horas en aparecer
2	**CodeQL**	tarda unos minutos en su primer análisis
3	**Sentry en tu código**	también temprano: tiene que pasar por tu pipeline hasta producción
4	**el monitor y la caída**	mientras esperás a los otros
5	**mergear el pull request de Dependabot**	cuando lo abra, y tu pipeline dé verde
6	**pausar el monitor, y la release**	al final

Empezá por lo que tarda: mientras esperás, hacés lo demás

Cierre — con esta clase termina el contenido

1. **TP9: las tres protecciones, el monitor, y tu aplicación observada**

   Empezá por Dependabot, que es lo que más tarda

2. **La defensa de los prácticos 5 a 9**

   Con tu QA y tu producción vivos, y preprod destruido y listo para nacer

3. **Tené el monitor y Sentry a mano**

   El monitor pausado, con el DOWN y el UP · si pasó un mes, repetí la caída y el error